



Шакиров Тимур Эдуардович

Fullstack-разработчик. Языковые модели в продуктовых системах.

20 лет · Санкт-Петербург · удалённо +7 981 691-94-36 edtimyr@gmail.com @TimaxLacs

github.com/TimaxLacs github.com/TimaxHack github.com/timaxlax npmjs.com/~timax

КРАТКО

Инженер, который встраивает языковые модели в работающие продукты. Спроектировал и написал первую версию LLM-шлюза - маршрутизация между поставщиками, фолбэк, биллинг по токенам, - через него идут все запросы коммерческого агрегатора нейросетей. Последний год по контракту веду e-commerce платформу: витрина, каталог, двуязычные карточки, конвейер наполнения с очередями генерации, развёртывание. Увлекаюсь оркестрацией и внедрением нейросетей - языковых моделей и компьютерного зрения - в разные проекты и области жизни, в том числе для автоматизации.

Образование - Санкт-Петербургский политехнический университет Петра Великого, техническая специальность; институт СПО СПбПУ, базовый инженерно-технический курс. Дальше ушёл в продуктовую разработку: программирую с 2021 года, с тех пор работаю с разными командами над отдельными проектами. Коммерческие проекты веду с 2024-го, всё описанное ниже сделано на практике и открыто для проверки по репозиториям.

НАВЫКИ

ЯЗЫКИ	Python, TypeScript, JavaScript
ФРОНТЕНД	React, Vite, Tailwind, Capacitor (iOS и Android)
БЭКЕНД	Node.js, Express, FastAPI, GraphQL (Hasura), REST, PostgreSQL / PostGIS, Redis, SQLite, MinIO
ИИ	маршрутизация между поставщиками и фолбэк, биллинг токенов, очереди генерации текста и изображений, RAG и MCP, распознавание речи
ПРОД	Docker, nginx, systemd, Linux, Git, CI, настройка DNS и почты
АНГЛИЙСКИЙ	технический и письменный, профессиональный уровень

РАБОТА С МОДЕЛЯМИ

С 2024 года языковые модели входят в ежедневную разработку. Текущие среды: Cursor, Claude Code, ClawCode. Автокомплит использую постоянно. Исследование, ограничения и критерии приёмки сначала фиксирую сам, затем наполняю решение самостоятельно или с моделью.

Перед работой с моделью записываю в репозиторий полный набор правил: как гонять проверки, чего не трогать. Те же ограничения дублирую в CI и хуках, чтобы их нельзя было обойти. Код с моделью пишу узкими шагами и через отдельных субагентов: план, правка, прогон тестов и линтеров.

Цикл исполняют агенты из [cursor-agent-workflows](#): анализ, план, реализация и проверка разведены; проверка идёт отдельно и не принимает код, который только что написал исполнитель. Скоуп файлов фиксирую заранее, параллельные правки если и делаю, то изолирую. Тесты, которые модель написала вместе с реализацией, не считаю достаточным доказательством: смотрю негативные и граничные случаи, секреты и зависимости. Написанный код всегда проверяю сам и без своего ревью не отдаю.

ОПЫТ РАБОТЫ

Deep.Assistant - агрегатор нейросетей

2024-2025

Основатель, backend и DevOps · команда из трёх человек · @DeepGPTBot, канал @gptDeep - 1 553 подписчика

Коммерческий продукт в работе: чат с моделями, генерация изображений, музыки и голоса, API-шлюз, мини-приложения в Telegram и VK.

LLM-шлюз - github.com/deep-assistant/api-gateway

Спроектировал и написал первую версию; шлюз вырос из [resale-chatgpt-azure](#) в инфраструктуру всех запросов продукта. Основной автор: HTTP-сервер, конфигурация моделей, менеджер токенов, слой работы с базой; веду репозиторий. Около 37 моделей от шести поставщиков за одним OpenAI-совместимым интерфейсом.

- **Таблица деградации вместо простого ретрая:** у каждой модели цепочка из 12-16 замен, подобранных так, чтобы ответ оставался пригодным для того же сценария. Недоступность одного поставщика не останавливает продукт.
- **Единый счёт поверх несопоставимых прайсов:** у каждой модели свой коэффициент пересчёта токенов во внутреннюю валюту, от 1.7 до 40. Продукт считает себестоимость запроса и наценку.
- **Идемпотентные переводы между счётами:** эксклюзивные блокировки с распознаванием зависших, атомарная запись, восстановление незавершённых операций при старте.

- **Скорость подключения моделей:** o1-preview и o1-mini – 27 сентября 2024, deepseek-reasoner – 30 января 2025, сразу с тарификацией и фолбэком.

Telegram-бот - github.com/deep-assistant/telegram-bot

Мои части: переводы внутренней валюты на конечном автомате (модуль с нуля), синхронизация пользователей с ограничением параллелизма под лимиты Telegram, транскрибация аудио с отдельным коэффициентом тарификации, оплата через Telegram Stars, окружения TEST и PROD.

Е-commerce платформа (проект заказчика, под NDA)

декабрь 2025 – настоящее время

Контрактная разработка: фронтенд, гео-модуль, мобильные сборки, смежный бэкэнд и развёртывание

Маркетплейс с каталогом и картами: React, TypeScript, Hasura GraphQL, восемь сервисов, PostgreSQL / PostGIS, Redis, MinIO, Docker, Capacitor. Веду клиентское приложение и участвую в проектировании; на бэкэнде – платежи, баланс, обработчики событий. Детали продукта под NDA, о модулях рассказываю свободно.

- **Витрина:** каталог, карточка товара, поиск с автодополнением, фильтры, регистрация нескольких типов аккаунтов с MFA, библиотека UI-компонентов.
- **Двухязычность RU/EN и автоперевод карточек:** словари вынесены из компонентов, перевод идёт через хук, серверный роут и CLI-скрипт, прогоняющий каталог пачкой.
- **Конвейер наполнения каталога:** источники за общим контрактом, дедупликация по внешнему id, штрихкоду и отпечатку характеристик. Нейросети работают после сбора отдельными очередями – текст и изображения. Факты карточки модель не переписывает, иначе ломается дедупликация; упавший воркер оставляет товар на витрине с исходными данными.
- **Кошелёк и заказы:** сервисы баланса и платежей, внешний эквайринг, обработчики смены статуса. Сборки Capacitor под iOS и Android, выкладка через Docker и systemd.

Гео-модуль

Единый интерфейс поверх Яндекс.Карт, Google Maps и 2ГИС. Компоненты карты, маркера и зоны не знают о поставщике: движки принимают команду и передают её адаптеру, адаптер переводит унифицированные данные в вызовы своего SDK. Смена поставщика – один параметр конфигурации. Геометрия зон: буферы вдоль маршрута с переходами к круговым зонам маркеров через квадратичные кривые Безье, объединение полигонов с отверстиями и мультиполигонами, валидация, кэширование расчётов. Перетаскиваемые маркеры и пользовательские зоны; изменения с карты поднимаются в состояние приложения через колбэки. Геокодинг на серверном роуте, чтобы ключи API не попадали в браузер. Хранение в PostGIS, поиск по радиусу, смоук-тесты, документация по выносу модуля в самостоятельное приложение.

ManyAir – платформа международных денежных переводов

август 2025 – май 2026

Проект заказчика: фронтенд, смежный бэкэнд и сервер

Микросервисы вокруг Hasura GraphQL: router, обработчик событий, рассылка, Telegram-бот, генератор документов, курсы валют; PostgreSQL, Redis, MinIO. Устойчивая доставка уведомлений: при недоступном канале статус возвращает «не отправлено», приглашение не помечается доставленным. Изоляция данных между ролями, выкладка сервисов, настройка почты и DNS.

Matreshka – публичный сайт AI/e-commerce платформ

2026

Фронтенд и запуск сайта: React, Vite, TypeScript, Tailwind; nginx с SSL, DNS и почта. Второй проект для того же заказчика – matreshka.club.

КЕЙСЫ С ИИ · 2021-2026

- **Контент-конвейер с отдельной ИИ-модерацией.** Черновик пишет одна модель, вторая проверяет отдельным вызовом, замечания возвращаются на правку до трёх кругов, затем публикация; состояние в SQLite, чтобы пережить падение процесса. Генерация отделена от проверки, сбой не теряет данные – [ai-post-generator](#).
- **Reasoning-конвейер поверх обычных чат-моделей**, апрель 2025. Пять стадий: предсказание намерения, пошаговое решение, состязательная критика собственного ответа, доработка, синтез; вариант на LangChain с Tree of Thoughts. Встроенный режим рассуждений не используется – поведение воспроизводится оркестрацией промптов – [test-resorning](#).
- **Свой инструментарий для работы с моделями.** Self-hosted LightRAG и MCP-сервер к нему: режимы поиска, ранкинг, ответы со ссылками ([lightrag-mcp](#)). 1 900 строк спецификаций агентных процессов: конвейер безопасного изменения кода из пяти агентов, бутстрап проекта, процесс исследования ([cursor-agent-workflows](#)). Пользуюсь ежедневно.
- **ai-lector**, 2025 – бот, перерабатывающий аудиолекции под нужный формат. Собрал локальный Telegram Bot API server через stake, чтобы обойти лимит на размер файла.
- **vk-tg-parser**, 2026 – rip-пакет: единый парсер Telegram через MTPROTO и VK через API с общей моделью данных и мониторингом в реальном времени.
- **Компьютерное зрение**, 2021 – бот замерзания Онежского озера: уровень замерзания по кадрам с уличной камеры – github.com/TimaxHack/ghj.

ОТКРЫТЫЙ КОД · С 2022

deep.foundation

2022 — настоящее время

Участник организации · github.com/deep-foundation

Экосистема ассоциативного хранилища. Пакеты ИИ-тулинга в официальном скоупе: @deep-foundation/ai, ai-models, ai-templates, ai-tasks и соседние. Свой пакет [deep-files-sync](#) (13 версий) проецирует граф связей на файловую систему: содержимое базы читается и правится через `grep`, редактор и `git`.

- **InnerEcho** - локальный голосовой ассистент: распознавание речи, языковая модель, синтез с клонированием голоса; npm-клиенты синтеза `xtts-v2-js` и `zonosjs` - [InnerEcho](#).
- **tg-runner** - оркестрация Telegram-ботов: конечный автомат с Redis, воркер в Docker, SDK и CLI - [tg-runner-orchestrator](#).

26 npm-пакетов - npmjs.com/~timax · полное портфолио - timaxlacs.github.io